Dominando Django

Modelos, ORM, Vistas Basadas en Clases y APIs

// Una guía visual y práctica de arquitectura backend

Gemini Notebook

Node 1 (Left)

Model (Datos)



Define la estructura de la base de datos.

Node 2 (Center)

View (Lógica)



El 'cerebro'. Procesa peticiones HTTP y consulta datos.

Node 3 (Right)

Template (UI)



Presentación visual (No necesario para APIs puras).

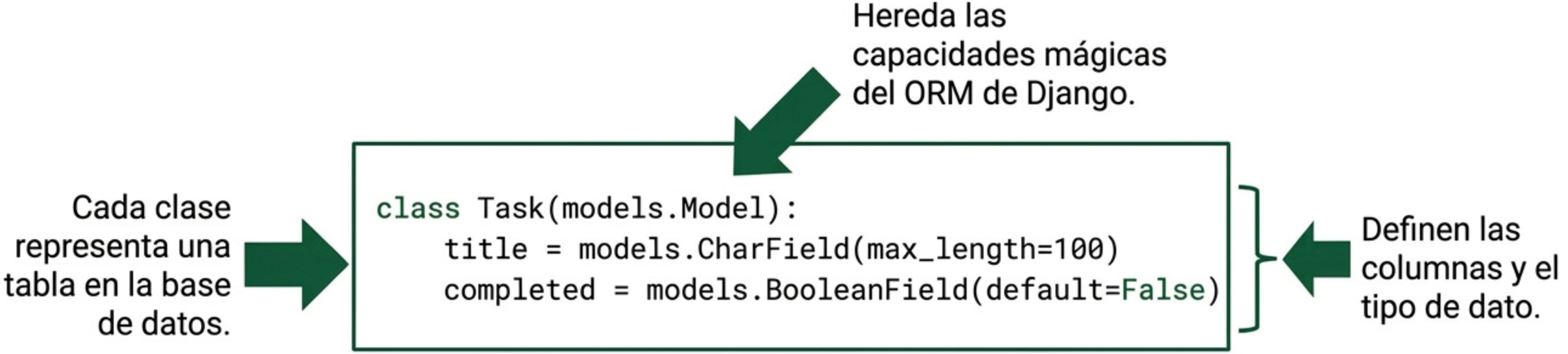
Gemini Notebook



Object-Relational Mapping (ORM): Un traductor automático. Permite interactuar con la base de datos usando clases y métodos de Python en lugar de escribir consultas SQL complejas.

Gemini Notebook

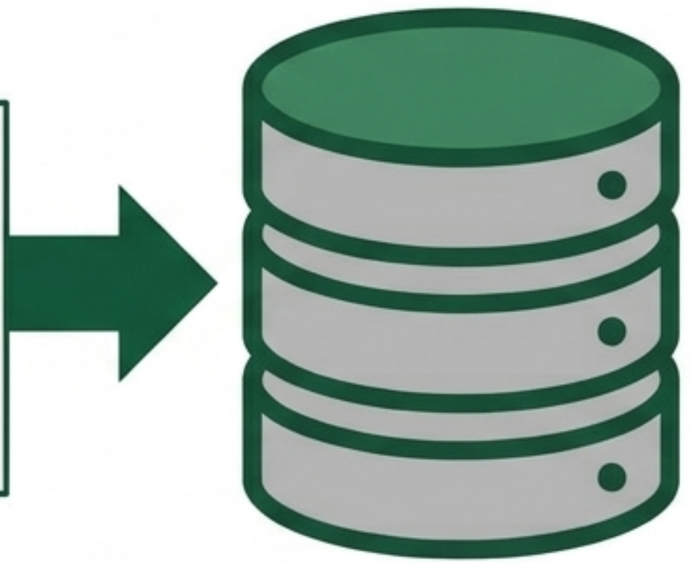
Anatomy of a Model



Gemini Notebook

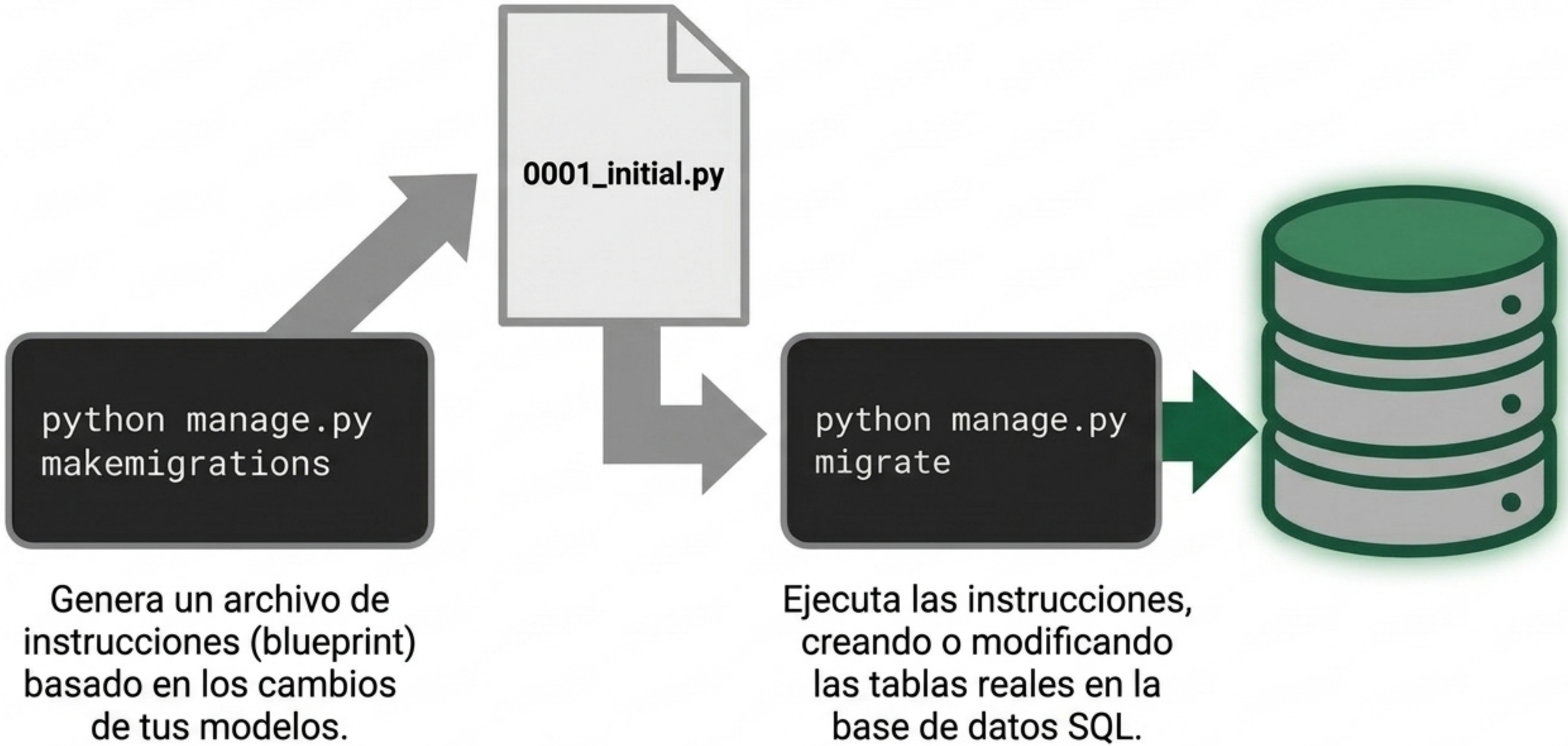
settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```



El ORM es **agnóstico**. Cambiando el ENGINE en settings.py, puedes migrar de SQLite a PostgreSQL o MySQL sin alterar una sola línea de tus Modelos.

Gemini Notebook



Gemini Notebook

El poder de los QuerySets

Crear (Create)

```
Task.objects.create(title='Aprender Django')
```

Leer (Read)

```
Task.objects.all() \
Task.objects.filter(completed=False)
```

Actualizar (Update)

```
task = Task.objects.get(id=1) \
task.completed = True \
task.save()
```

Eliminar (Delete)

```
Task.objects.get(id=1).delete()
```

El Manager '**objects**' es la interfaz principal para interactuar con la base de datos de forma limpia y orientada a objetos.

Gemini Notebook

Backend



Modelos



ORM





Clientes







¿Qué es una **API**? Es el contrato de comunicación. Las aplicaciones modernas no consumen HTML, consumen datos puros en formato JSON *(JavaScript Object Notation)*. Django debe traducir sus modelos a JSON.

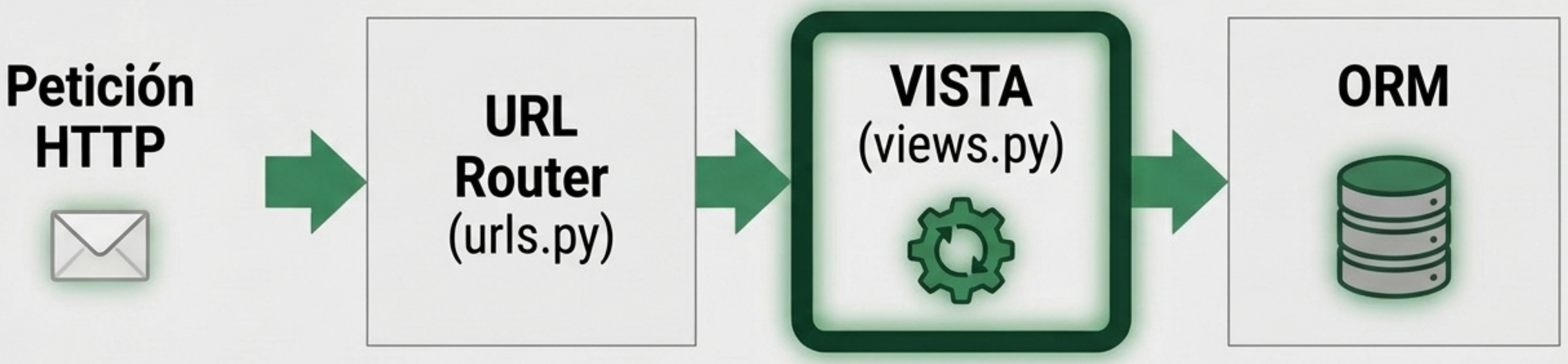
Gemini Notebook

Protocolo HTTP y Endpoints

MÉTODO	ENDPOINT	ACCIÓN
GET	`/api/tasks/`	Leer (Lista de tareas)
POST	`/api/tasks/`	Crear (Nueva tarea)
DELETE	`/api/tasks/<id>/`	Eliminar (Tarea específica)

Un endpoint es la URL expuesta; el método HTTP define la intención de la petición.

Gemini Notebook



La Vista es el controlador. Recibe la petición HTTP, decide qué datos pedirle al ORM, y formatea la respuesta que se enviará de vuelta al cliente.

Gemini Notebook

FBV (Funciones) vs CBV (Clases)

Vistas Basadas en Funciones (FBV)	Vistas Basadas en Clases (CBV)
• Más código repetitivo (if request.method == 'POST'). • Difícil de extender o reutilizar. • Flujo de ejecución explícito y fácil de leer para principiantes.	• Estructura orientada a objetos (Herencia). • Código extremadamente conciso (Reutilización). • El flujo interno está abstraído (“Magia”).

Las CBVs son el estándar de la industria en Django por su capacidad para escalar y evitar la repetición de código.

Gemini Notebook

Anatomy of a View (CBV)

Hereda de la clase base View.

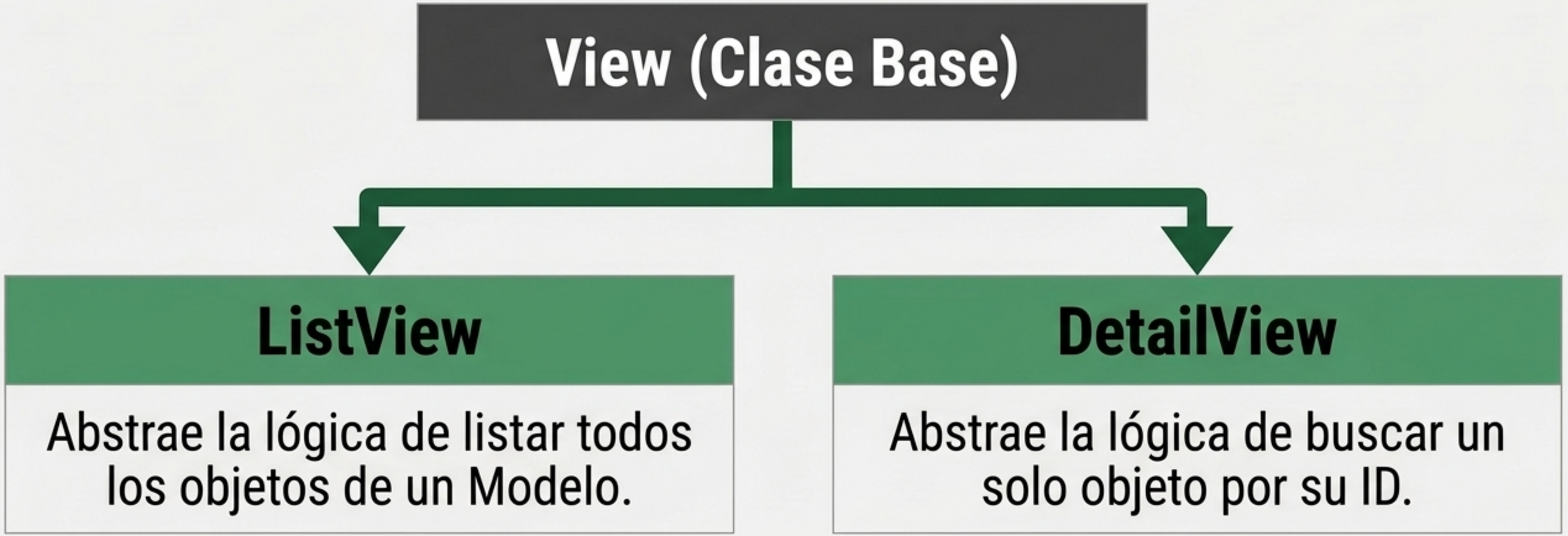
```
class TaskView(View):
    def get(self, request):
        # Lógica de lectura
        return JsonResponse(...)

    def post(self, request):
        # Lógica de creación
        return JsonResponse(...)
```

El método HTTP se mapea automáticamente a un método de clase con el mismo nombre. ¡Adiós a los if request.method!

Gemini Notebook

Inheritance Hierarchy: Generic Views (CBV)



Django puro incluye variantes genéricas que reducen la lógica de lectura a solo dos líneas de código: definir el model y el template_name.

Gemini Notebook

Django REST Framework (DRF)



- Por qué las CBVs estándar no bastan: Django puro está diseñado para retornar HTML.
- DRF proporciona CBVs especializadas para serializar objetos a JSON, manejar autenticación y validar datos.
- Es el estándar de oro para crear APIs web en Django.

Gemini Notebook

Serializer (Convertir Objetos a JSON)




```python
class TaskSerializer(serializers.ModelSerializer):
    class Meta:
        model = Task
        fields = '__all__'
```

El Serializador automatiza la conversión bidireccional: transforma instancias del ORM en JSON para respuestas, y valida datos JSON entrantes para crear instancias del ORM.

Gemini Notebook



Elige la variante según la complejidad de tu lógica de negocio.
Para operaciones estándar, usa Generics.

Gemini Notebook

Práctica: Endpoint de Creación (POST)

HTTP
POST
JSON

```python
from rest_framework import generics

class TaskCreate(generics.CreateAPIView):
    querysot = Task.objects.all()
    serializer_class = TaskSerializer
```

Esta variante genérica expone exclusivamente un método POST.

Utiliza el serializador para validar y guardar automáticamente los datos entrantes en la DB.

Gemini Notebook

Práctica: Endpoint de Eliminación (DELETE)

HTTP
DELETE
/1/

```python
from rest_framework import generics

class TaskDestroy(generics.DestroyAPIView):
    querysot = Task.objects.all()
    serializer_class = TaskSerializer
```

```python
path('tasks/<int:pk>/', TaskDestroy.as_view())
```

Expone exclusivamente el método DELETE.

La vista busca automáticamente en el queryset el objeto con el ID (pk) proporcionado en la URL y lo elimina.

Gemini Notebook

La Variante Definitiva: ViewSets

1 Clase

```python
class TaskViewSet(viewsets.ModelViewSet):
    queryset = Task.objects.all()
    serializer_class = TaskSerializer
```

=

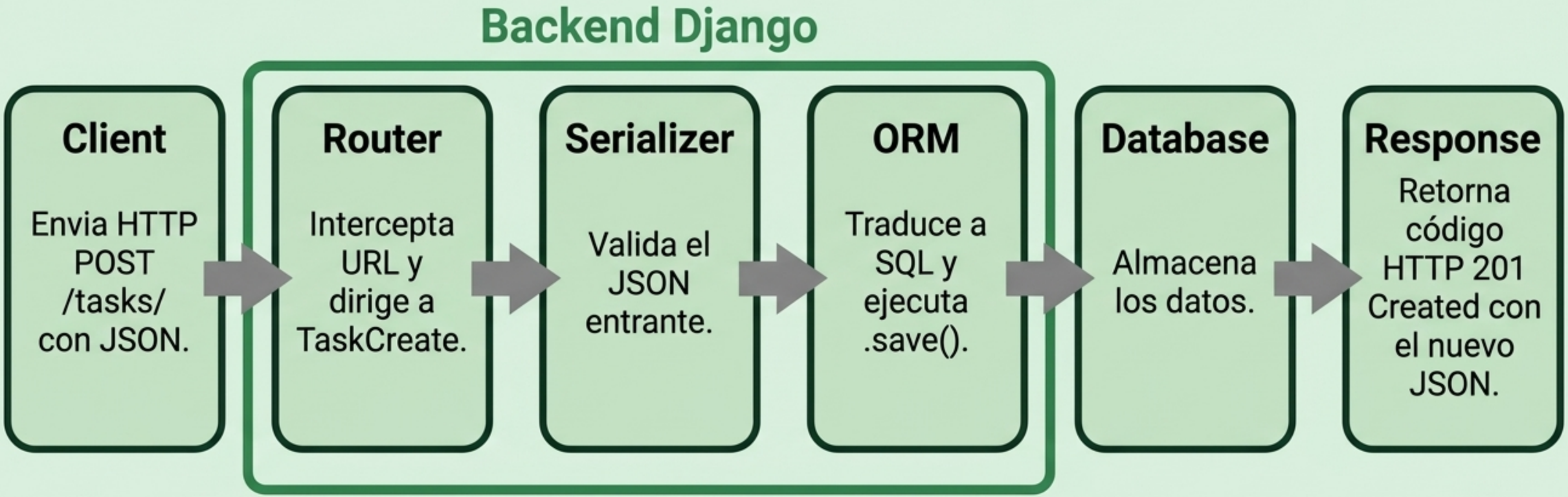
6 Endpoints

`GET /tasks/`	(Listar)
`POST /tasks/`	(Crear)
`GET /tasks/1/`	(Leer)
`PUT /tasks/1/`	(Actualizar)
`DELETE /tasks/1/`	(Eliminar)
`OPTIONS /tasks/`	(Metadatos)

ModelViewSet consolida toda la lógica CRUD en 3 líneas de código, manejando las peticiones y respuestas mediante Routers.

Gemini Notebook

El Ciclo de Vida de una Petición



Gemini Notebook

Principios del Backend en Django

1. **Piensa en Datos, no en SQL**: Diseña Modelos robustos; deja que el ORM haga el trabajo pesado de las migraciones y consultas.

2. **Aprovecha la Herencia**: Usa Vistas Basadas en Clases (CBVs). No reinventes la rueda si un GenericView o ViewSet ya resuelve el problema.

3. **Delega en el Serializador**: Mantén tus Vistas 'delgadas'. La validación de datos y las reglas de negocio complejas pertenecen a los Serializadores y Modelos.

// Un sistema modular, predecible y escrito con Clean Code.

Gemini Notebook